CS & IT ENGINEERING

Operating System

Process Synchronization

Lecture - 03

By- Vishvadeep Gothi sir

Recap of Previous Lecture

- **Topic** Mutual Exclusion
- **Topic** Progress
- **Topic** Bounded Waiting
- **Topic** Two-Process Solution for Critical Section

Topics to be Covered

- Topic: Peterson's Solution
- Topic: Hardware Solutions of Synchronization
- Topic: Test-And-Set()
- Topic: Swap()

Solution 1

```
Boolean lock=false;
```

P0

```
while(true)
{
    while(lock);
    lock=true;
        //CS
    lock=false;
    RS;
}
```

P1

```
while(true)
{
    while(lock);
    lock=true;
        //CS
    lock=false;
    RS;
}
```

Solution 2

```
int turn=0;

while(true)
{
    while(turn!=0);
    CS
    turn=1;
    RS;
}
```

```
while(true)
{
    while(turn!=1);
    CS
    turn=0;
    RS;
}
```

Peterson's Solution

shows intension of a process if it wants to enter into critical section

```
Boolean Flag[2]={False, False};
int turn;
```

turn → for giving priority to a process

P0

```
while(true) {
    Flag[0]=true;
    turn=1;
    while(Flag[1] && turn==1);
        CS
    Flag[0]=False;
        RS;
}
```

P1

```
while(true){
    Flag[1]=true;
    turn=0;
    while(Flag[0] && turn==0);
        CS
    Flag[1]=False;
        RS;
}
```

- ✓ Mutual Exclusion
- ✓ Progress
- ✓ Bounded waiting

Topic : Synchronization Hardware

1. TestAndSet()
2. Swap()

some instns were introduced in Instn set architecture (ISA) to provide synchronization

Topic : TestAndSet()

Returns the current value flag and sets it to true.

Topic : TestAndSet()

```
Boolean Lock=False;

boolean TestAndSet(Boolean *trg){
boolean rv = *trg;
*trg = True;
Return  rv;
}
```

shared among all processes

P0 P1

```
while(true)
{
  while(TestAndSet(&Lock));
        CS
  Lock=False;
}
```

Lock = ~~F T~~ ~~F T~~ ~~F T~~

✓ Mutual Exclusion

✓ Progress

✗ Bounded waiting

Topic : Swap()

local for each process (not shared)

```
Boolean Key;                    //Local
Boolean Lock=False;             //Shared

void Swap(Boolean *a, Boolean *b)
{
boolean temp = *a;
*a=*b;
*b=temp;
}
```

✓ Mutual Exclusion
✓ progress
✗ Bounded waiting

P0 : $key_0 = F\,F\,F$

P1 : $key_1 = F\,F\,T$

$lock = F\,F\,F\,F\,F\,T$

```
while(true){
  Key = True;
  while (key==True)
        Swap(&Lock, &Key);
     CS
  Lock=False;
}
```

2 mins Summary

Topic Peterson’s Solution

Topic Hardware Solutions of Synchronization

Topic Test-And-Set()

Topic Swap()

Happy Learning

THANK - YOU